

PIA - Proyecto Clasificación de Basura

Alberto Piedra Venegas

February 2026

Índice

1. Planteamiento del problema	2
2. Análisis del dataset	2
3. Modelo Fully Connected	3
3.1. Estrategias para combatir el Overfitting y el desbalance de clases	3
3.2. Resultados	4
4. Modelo CNN	4
4.1. Estrategias para combatir el Overfitting y el desbalance de clases	5
4.2. Resultados	5
5. Modelo Transfer Learning	6
5.1. Estrategias para combatir el Overfitting y el desbalance de clases	8
5.2. Resultados	8
6. Comparación entre los modelos	9
7. Conclusiones	10

1. Planteamiento del problema

Uno de los aspectos más importantes para el cuidado del medio ambiente es el correcto manejo de los residuos, por lo que es fundamental contar con sistemas que permitan identificar y separar correctamente los diferentes tipos de basura. En este proyecto, nos enfocaremos en el desarrollo de un sistema de clasificación de basura que permita identificar y separar correctamente los diferentes tipos de basura.

Para utilizar estos modelos en la vida real, se podría colocar una cámara en el exterior de un cubo de basura que detecte el tipo de basura que entra y a posteriori se abra la tapa correspondiente para que el usuario deposite la basura en el cubo correcto.

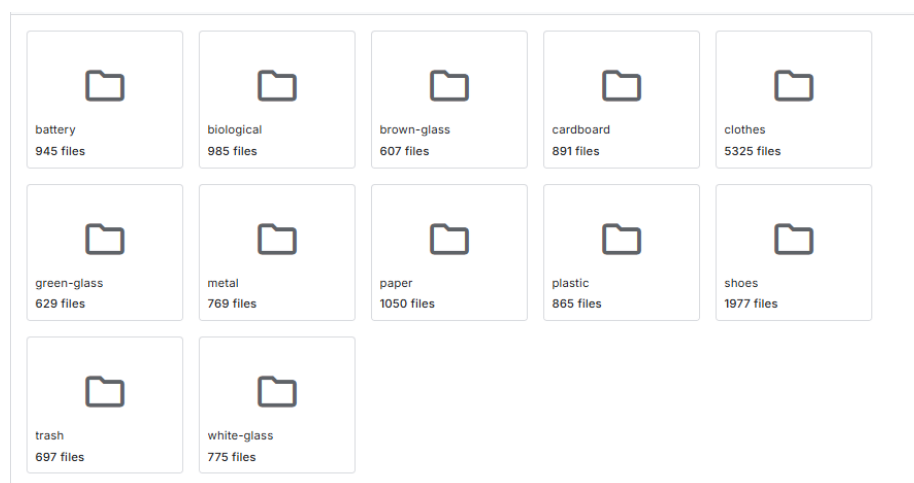
2. Análisis del dataset

Para este proyecto utilizaremos un dataset sacado de la plataforma Kaggle, en este dataset encontraremos más de 15000 imágenes de entre 12 tipos distintos de basura.

Los tipos a los que pertenece cada imagen son los siguientes:

- | | | |
|-----------------|----------------|-----------------|
| ■ Bateria | ■ Ropas | ■ Plastico |
| ■ Biologico | ■ Vidrio verde | ■ Zapatos |
| ■ Vidrio marrón | ■ Metal | ■ Basura |
| ■ Cartón | ■ Papel | ■ Vidrio blanco |

Este dataset cuenta con un pequeño desbalance de clases con el que lidiaremos mediante una función de pesos que se encargará de penalizar más a las clases con menos imágenes.



3. Modelo Fully Connected

El primer modelo que utilizaremos será un modelo denso o también llamado Fully Connected. Este modelo es el más sencillo de los tres, que se utilizarán para este proyecto de clasificación.

Para este modelo primero escalaremos las imágenes a un tamaño de 224x224 píxeles, luego dividiremos el valor de los píxeles entre 255 para que se encuentren en un rango de 0 a 1. A continuación, aplanaremos las imágenes para que puedan ser procesadas por el modelo denso. Este modelo constará de 3 capas densas con una función de activación ReLU y una capa de salida con una función de activación Softmax, con 12 neuronas cada una representando cada una de las clases.

```
1  modeloFullyConnected = tf.keras.Sequential()
2
3
4  modeloFullyConnected.add(tf.keras.layers.Rescaling(1./255,
5  input_shape=(224, 224, 3)))
6  modeloFullyConnected.add(tf.keras.layers.Flatten(input_shape
7  =(224, 224,3)))
8  modeloFullyConnected.add(tf.keras.layers.Dense(128, activation='
9  relu'))
10 modeloFullyConnected.add(tf.keras.layers.Dropout(0.3))
11 modeloFullyConnected.add(tf.keras.layers.Dense(64, activation='
12 relu'))
13 modeloFullyConnected.add(tf.keras.layers.Dense(12, activation='
14 softmax'))
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
```

Entrenaremos nuestro modelo con los datos de entrenamiento y validación, para después evaluar su rendimiento con los datos de prueba.

3.1. Estrategias para combatir el Overfitting y el desbalance de clases

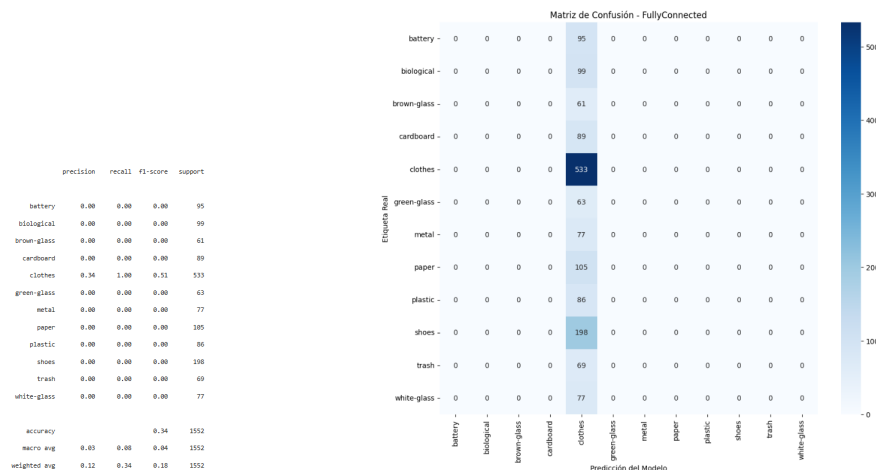
Para combatir el Overfitting utilizaremos la técnica del Dropout que consiste en desactivar aleatoriamente el 30% de las neuronas de una capa durante el entrenamiento.

Además utilizaremos una función de pesos para combatir el desbalance de clases. Esta función se encarga de penalizar más a las clases con menos imágenes.

Y por último añadiremos un EarlyStopping y un callback para reducir el learning rate cuando el modelo deje de mejorar y para detener el entrenamiento si el modelo no mejora durante unas épocas determinadas..

3.2. Resultados

Tras entrenar el modelo utilizando las estrategias mencionadas anteriormente, obtuvimos los siguientes resultados utilizando los datos de prueba:



Como se puede ver en los resultados, el modelo no da muy buenos resultados y solo predice una misma categoria que es ropas.

Esto es debido a la perdida de capacidad de discriminación ya que tienen una gran cantidad de parametros ($224 \times 224 \times 3 = 150528$) y a la función de flatten, el modelo prefiere memorizar la clase que mas haya (clothes) y predecir todas las imagenes de este tipo sin aprender ninguna característica ya que asi sería la que "mejor resultado" le daría. Sin embargo esto hace que cuando se utilicen imágenes nuevas para probar el modelo, este falle mucho.

4. Modelo CNN

El modelo CNN es un modelo más avanzado que el modelo Fully Connected, ya que utiliza capas convolucionales para extraer características de las imagenes. Para nuestro modelo, contaremos con 10 capas distintas:

- La primera de ellas servira como capa de entrada y se encargará también de escalar las imagenes para que los pixeles pasen de un rango de 0 a 255 a un rango de 0 a 1.
- Luego contaremos con 2 bloques de capas convolucionales con una funcion de activacion ReLU, esta capa se encargará de utilizar filtros para detectar distintos patrones y bordes.
- También contaremos con capas de agrupamiento Max Pooling que se encargarán de reducir el tamaño de las imagenes para reducir el computo de la imagen.

- Tendremos una capa con el metodo `flatten()` que se encargará de aplanar las imagenes para que puedan ser procesadas por el modelo denso.
- Tendremos dos capas densas con una funcion de activacion ReLU y una capa de salida con una funcion de activacion Softmax, con 12 neuronas cada una representando cada una de las clases.

```

1  modeloCNN = tf.keras.Sequential()
2
3
4  modeloCNN.add(tf.keras.layers.Rescaling(1./255, input_shape
5  =(224, 224, 3)))
6  modeloCNN.add(Conv2D(filters=32, kernel_size=(3, 3), activation
7  ='relu', input_shape=(224, 224, 3)))
8  modeloCNN.add(MaxPool2D(pool_size=(2, 2)))
9  modeloCNN.add(tf.keras.layers.Dropout(0.5))
10 modeloCNN.add(Conv2D(filters=64, kernel_size=(3, 3), activation
11 ='relu'))
12 modeloCNN.add(MaxPool2D(pool_size=(2, 2)))
13 modeloCNN.add(Flatten())
14 modeloCNN.add(Dense(128, activation='relu'))
15 modeloCNN.add(tf.keras.layers.Dropout(0.5))
16 modeloCNN.add(Dense(12, activation='softmax'))
17
18 modeloCNN.compile(optimizer='adam', loss='
19 sparse_categorical_crossentropy', metrics=['accuracy'])
20
21 historyCNN = modeloCNN.fit(X_train, y_train, epochs=20,
22 validation_data=(X_val, y_val), class_weight=dict_weights,
23 callbacks = [callback, reduce_lr])
24
25 modeloCNN.evaluate(X_test, y_test)

```

Entrenaremos nuestro modelo con los datos de entrenamiento y validación, para después evaluar su rendimiento con los datos de prueba.

4.1. Estrategias para combatir el Overfitting y el desbalance de clases

Para combatir el Overfitting utilizaremos la tecnica del Dropout en 2 capas distintas, con una tasa de 0.5 ambas, estas capas se encargarán de desactivar aleatoriamente el 50 % de las neuronas de la capa durante el entrenamiento.

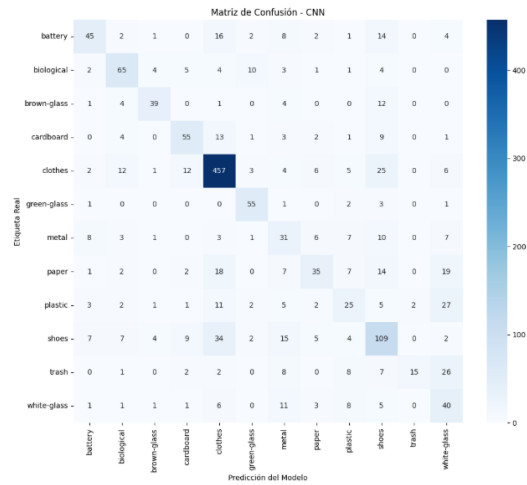
Además utilizaremos una función de pesos para combatir el desbalance de clases. Esta función se encarga de penalizar más a las clases con menos imágenes.

Y por último añadiremos un EarlyStopping y un callback para reducir el learning rate cuando el modelo deje de mejorar y para detener el entrenamiento si el modelo no mejora durante unas epocas determinadas..

4.2. Resultados

Tras entrenar el modelo utilizando las estrategias mencionadas anteriormente, obtuvimos los siguientes resultados utilizando los datos de prueba:

	precision	recall	f1-score	support
battery	0.63	0.47	0.54	95
biological	0.63	0.66	0.64	99
brown-glass	0.75	0.64	0.69	61
cardboard	0.63	0.62	0.62	89
clothes	0.81	0.86	0.83	533
green-glass	0.72	0.87	0.79	63
metal	0.31	0.48	0.35	77
paper	0.56	0.33	0.42	105
plastic	0.36	0.29	0.32	86
shoes	0.58	0.55	0.53	198
trash	0.88	0.22	0.35	69
white-glass	0.38	0.52	0.38	77
accuracy			0.63	1552
macro avg	0.59	0.54	0.54	1552
weighted avg	0.64	0.63	0.62	1552



Como se puede ver en los resultados el modelo de CNN da mejores resultados que el modelo Fully Connected, ya que tiene una mayor capacidad de discriminación y puede aprender características más complejas de las imágenes.

Aun así los resultados no son tan buenos como nos gustaría ya que solo tiene un accuracy de un 63 % y esto es debido a varias razones:

- Desbalanceo de datos: Existe un gran desbalanceo de datos entre algunas categorías como por ejemplo ropas (+500 imágenes) y otras como por ejemplo vidrio verde (-100 imágenes).
- Confusión entre categorías: El modelo confunde muchas categorías entre sí, como por ejemplo cartón con papel, plástico con vidrio, etc.

El modelo se tiende a inclinar hacia las clases mayoritarias. Aunque el Weighted Avg (promedio ponderado) es 0.64, el Macro Avg (que da el mismo peso a todas las clases) es de 0.54. Esto indica que si usaras este modelo en un entorno real con una distribución equitativa de basura, fallaría casi la mitad de las veces.

5. Modelo Transfer Learning

El modelo Transfer Learning consiste en utilizar un modelo pre-entrenado en un conjunto de datos grande y utilizarlo como base para nuestro modelo.

Para este proyecto utilizaremos el modelo EfficientNetB1 pre-entrenado en el conjunto de datos ImageNet. Este modelo es una red neuronal convolucional que ha sido entrenada en más de 14 millones de imágenes y ha obtenido una precisión del 84 % en el conjunto de datos ImageNet.

Primero cargaremos el modelo pre-entrenado y le añadiremos el parámetro `include_top=False` para que no cargue la capa de salida.

Luego tendremos dos etapas distintas en nuestro modelo:

1. Fase de Calentamiento: En esta fase congelamos el conocimiento de EfficientNetB1 y entrenaremos solo las capas que añadamos nosotros.
2. Fase de Fine-Tuning: En esta fase descongelaremos solo las últimas 40 capas para que el modelo se adapte a nuestro dataset. Aquí también añadiremos capas de GlobalAveragePooling2D para reducir la complejidad de los datos y 2 capas densas, una para razonamiento intermedio y otra con el clasificador final del modelo.

```
1
2 base_model = EfficientNetB1(weights='imagenet', include_top=
  False, input_shape=(224, 224, 3))
3 base_model.trainable = False
4
5 inputs = tf.keras.Input(shape=(224, 224, 3))
6 x = base_model(inputs, training=False)
7 x = GlobalAveragePooling2D()(x)
8 x = Dense(256, activation='relu')(x)
9 x = Dropout(0.4)(x)
10
11 predictions = Dense(len(np.unique(y_train)), activation='
  softmax')(x)
12 modeloB1 = Model(inputs, predictions)
13
14 modeloB1.compile(optimizer=Adam(learning_rate=1e-3),
15                  loss='sparse_categorical_crossentropy',
16                  metrics=['accuracy'])
17
18 history_warmup = modeloB1.fit(
19     X_train, y_train,
20     epochs=10,
21     validation_data=(X_val, y_val),
22     class_weight=dict_weights,
23     batch_size=32,
24     callbacks=[callback, reduce_lr]
25 )
26
27 base_model.trainable = True
28 for layer in base_model.layers[:-40]:
29     layer.trainable = False
30
31 modeloB1.compile(optimizer=Adam(learning_rate=1e-5),
32                  loss='sparse_categorical_crossentropy',
33                  metrics=['accuracy'])
34
35
36 historyTransferLearning = modeloB1.fit(
37     X_train, y_train,
38     epochs=15,
39     validation_data=(X_val, y_val),
40     class_weight=dict_weights,
41     batch_size=32,
42     callbacks=[callback, reduce_lr]
43 )
```

5.1. Estrategias para combatir el Overfitting y el desbalance de clases

Para combatir el Overfitting utilizaremos la tecnica del Dropout que consiste en desactivar aleatoriamente el 40% de las neuronas de una capa durante el entrenamiento.

Además utilizaremos una función de pesos para combatir el desbalance de clases. Esta función se encarga de penalizar más a las clases con menos imágenes.

Y por último añadiremos un EarlyStopping y un callback para reducir el learning rate cuando el modelo deje de mejorar y para detener el entrenamiento si el modelo no mejora durante unas epocas determinadas..

5.2. Resultados

Tras entrenar el modelo utilizando las estrategias mencionadas anteriormente, obtuvimos los siguientes resultados utilizando los datos de prueba:

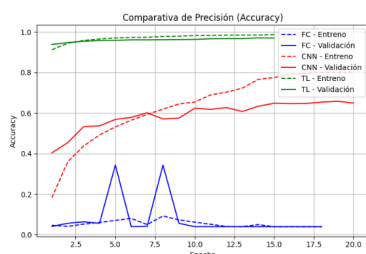


El modelo de Transfer Learning utilizando el EfficientNetB1 es el que mejores resultados da de los 3. Esto es debido a tres razones:

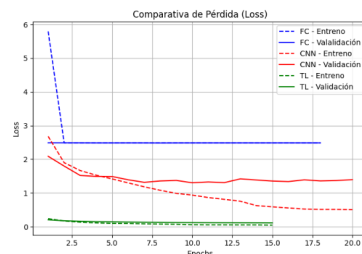
- Pre-conocimiento: Debido a que este modelo cuenta con conocimiento anterior basado en el dataset ImageNet, puede aprender características más complejas y generalizables.
- Fine-Tuning: A pesar de tener un modelo funcional, hacemos cambios para que se adapte mejor a nuestras categorías.
- Manejo de sobreajuste: Al tener las capas base congeladas, el modelo tiene muchísimos menos parámetros que aprender al principio. Esto evita que se sobreajuste o se aprenda de memoria el ruido de los datos.

6. Comparación entre los modelos

Aquí tenemos una comparación entre los tres modelos:



(a) Resultados Precisión



(b) Resultados Pérdida

Como se puede ver en las dos graficas podemos distinguir la calidad de los tres modelos, siendo el modelo denso el que peores resultados da, con un 0.1 de precisión y una perdida aproximada de 2,5, luego el modelo convolucional que mejora un poco respecto al anterior teniendo una precisión de 0.6 de media y la función de perdida teniendo un valor de 1.5. Por último el mejor modelo de todos el TransferLearning con un accuracy de 0.97 y una perdida de 0.1 de media.

El primer modelo obtiene tan malos resultados debido a la gran cantidad de paramtetros que llegan a las primeras neuronas además del uso de la función de flatten() que modifica los pixeles de entrada, aplanandolos y haciendo que pueda perder el contexto de que categoria de objeto es. Debido a esta perdida del contexto de los pixeles, el modelo llega a una conclusión que predecir la categoria que más imagenes tenga es la que mejor resultado va a producir ignorando todas las características de la imagen y no aprendiendo nada.

El modelo convolucional es una mejora respecto al modelo denso ya que no utiliza la función de flatten() para aplanar las imagenes al principio, haciendo que el modelo no pierda el contexto de los pixeles de la imagen. En cambio, utiliza Conv2D que se trata de unos filtros que recorren la imagen en buscar de patrones, bordes, texturas que permitan al modelo aprender las características de los distintos tipos. Estos filtros también permiten al modelo reconocer un tipo independientemente de en que parte de la imagen se encuentre. El aumento en el número de filtros de 32 a 64 hace que primero detecte rango más simples y luego detecta rasgos más complejos.

El modelo de TransferLearning da mucho mejor resultado que los otros dos modelos debido a dos razones principales:

- Conocimiento previo: El modelo de TransferLearning cuenta con un entrenamiento con millones de imagenes y ya sabe distinguir bordes, formas, texturas de algunas categorias mientras que la red convolucional y la densa empiezan de cero, haciendo que no puedan distinguir tan bien.
- Extracción de características robusta: Al ser una arquitectura mucho más profunda y optimizada, es capaz de detectar rasgos sutiles que a una CNN

pequeña se le escapan. Esto permite que el modelo no se confunda si la iluminación cambia o si el residuo está un poco deformado.

7. Conclusiones

El Transfer Learning es el modelo que mejores resultados da y es el unico que podría funcionar en un entorno real ya que es el que da unos resultados superiores al 90 %, el modelo CNN tienen un resultado aceptables, sin embargo no termina de poder diferenciar las pequeñas diferencias entre algunas categorías haciendo que su precisión disminuya de gran manera.

El modelo CNN se podría mejorar de dos maneras:

- Aumentar el número de filtros y capas para que pueda aprender más características de las imágenes, lo que haría que aumente su complejidad y por lo tanto su capacidad de generalización.
- Aumentar la cantidad de imágenes del dataset, esto se podría hacer añadiendo más imágenes de las categorías o con la utilización de técnicas de aumento de datos, como el aumento de la iluminación, rotación, etc.